
Utility-Diversity Aware Online Batch Selection for LLM Supervised Fine-tuning

Heming Zou¹ Yixiu Mao¹ Yun Qu¹ Qi Wang¹ Xiangyang Ji¹

Abstract

Supervised fine-tuning (SFT) is a commonly used technique to adapt large language models (LLMs) to downstream tasks. In practice, SFT on a full dataset is computationally expensive and sometimes suffers from overfitting or bias amplification. This facilitates the rise of data curation in SFT, which prioritizes the most valuable data to optimize. This work studies the online batch selection family that dynamically scores and filters samples during the training process. However, existing popular methods often (i) rely merely on the utility of data to select a subset while neglecting other crucial factors like diversity, (ii) rely on external resources such as reference models or validation sets, and (iii) incur extra training time over full-dataset training. To address these limitations, this work develops **UDS (Utility-Diversity Sampling)**, a framework for efficient online batch selection in SFT. UDS leverages the nuclear norm of the logits matrix to capture both data utility and intra-sample diversity, while estimating inter-sample diversity through efficient low-dimensional embedding comparisons with a lightweight memory buffer of historical samples. Such a design eliminates the need for external resources and unnecessary backpropagation, securing computational efficiency. Experiments on multiple benchmarks demonstrate that UDS consistently outperforms state-of-the-art online batch selection methods under varying data budgets, and significantly reduces training time compared to full-dataset fine-tuning. Code is available at <https://github.com/gfyddha/UDS>.

1. Introduction

The rapid progress of Large Language Models (LLMs) has reshaped natural language processing and enabled impressive generalization across a wide range of domains (Achiam et al., 2023; Brown et al., 2020; Liu et al., 2024; Touvron et al., 2023; Bai et al., 2023). To further improve their performance in specialized areas, Supervised Fine-tuning (SFT) has emerged as a typical post-training paradigm. However, training on a full dataset is not always advantageous. It entails substantial computational cost and has been shown to be less effective than using a small amount of carefully curated data (Albalak et al., 2024; Xia et al., 2024; Zhou et al., 2023; Xiao et al., 2026). These challenges highlight the importance of principled data selection strategies that can improve both efficiency and effectiveness in SFT.

In this work, we address this challenge by focusing on **on-line batch selection**, a paradigm that dynamically evaluates sample value and performs filtering during training (Wang et al., 2024; Loshchilov & Hutter, 2015; Katharopoulos & Fleuret, 2018; Mindermann et al., 2022). Instead of using all samples in a batch, the model assesses the importance of each sample as it is encountered and selects only a subset to participate in parameter updates. This approach enables the selection process to adapt in real time to the model’s current state and learning trajectory, improving both training efficiency and effectiveness.

However, despite promising progress, current online batch selection methods still face several issues that hinder their practical deployment. These methods typically rely solely on *data utility*, e.g., picking sample subsets with high loss (Loshchilov & Hutter, 2015; Jiang et al., 2019) or gradient magnitude (Katharopoulos & Fleuret, 2018). While such heuristics enable capturing the critical samples to reduce the current training loss, they evaluate sample value from a limited perspective. In practice, effective selection also requires considerations of *intra-sample diversity*, to encourage token-level exploration with fewer repetitive phrases within each training instance (Xie et al., 2020), and *inter-sample diversity* to suppress redundancy across examples by avoiding repeated training on near-duplicate content (Lee et al., 2021; Tirumala et al., 2023). Popular literature (Loshchilov & Hutter, 2015; Katharopoulos & Fleuret, 2018; Mindermann et al., 2022) prioritizes data utility and rarely examines the

¹Department of Automation, Tsinghua University. Correspondence to: Qi Wang <cheemswang@mail.tsinghua.edu.cn>, Xiangyang Ji <xyji@tsinghua.edu.cn>.

Table 1. Comparison of online batch selection methods under our desiderata (D1–D3). The columns denote: Data Utility, Intra-sample Div. (Intra-sample Diversity), Inter-sample Div. (Inter-sample Diversity), No External Res. (No External Resources), and Train. Time Reduc. (Training Time Reduction). The symbols denote: ✓ (support), and ✗ (don’t support).

Method	Data Utility	Intra-sample Div.	Inter-sample Div.	No External Res.	Train. Time Reduc.
Max Loss	✓	✗	✗	✓	✓
Max Grad	✓	✗	✗	✓	✗
RHO-Loss	✓	✗	✗	✗	✗
GREATS	✓	✗	✓	✗	✗
UDS (Ours)	✓	✓	✓	✓	✓

selection criteria through the lens of diversity.

Another primary issue involves both *external dependencies* and *computational overhead*. Some approaches (Mindermann et al., 2022; Wang et al., 2024) rely on a held-out validation set, which may be unavailable since we hardly know the exact distribution of test dataset. Other methods (Mindermann et al., 2022; Deng et al., 2023) use a reference model that is also shown to be impractical in the real world (Kaddour et al., 2023). Besides these external dependencies, several selection algorithms (Katharopoulos & Fleuret, 2018; Wang et al., 2024; Mindermann et al., 2022) introduce substantial computational costs that even exceed the expense of full-dataset training, raising efficiency concerns.

Based on the above evidence, this work proposes that an ideal online batch selection method should be comprehensive and satisfy the three desiderata (D1-D3) outlined below. A systematic comparison of representative methods against these criteria is summarized in Table 1.

D1: Jointly consider *data utility*, *intra-sample diversity*, and *inter-sample diversity*;

D2: Circumvent the access to external resources, e.g., reference model or validation set;

D3: Reduce the training time of the overall pipeline relative to the full-dataset SFT.

Leveraging logits as basis for online scoring. To operationalize these desiderata, a key challenge lies in how to efficiently and reliably score candidate samples during training. In online batch selection, each candidate sample must be evaluated under the current model to quantify its contribution to learning. This evaluation typically requires at least a forward pass, as it can fully capture how the model presently interprets the sample and thus provide an accurate basis for selection (Huang et al., 2023; Shorinwa et al., 2025). To achieve this efficiently (D3) without external resources (D2), we must avoid expensive gradient computations for each candidate sample and leverage intrinsic signals already available during forward propagation. These considerations collectively justify leveraging the model’s output logits, which are naturally produced during the forward pass and encode rich information about both sample utility and diversity (Qiu

& Miikkulainen, 2024; Geng et al., 2023). Specifically, we compute an *intra-sample importance score* based on the nuclear norm of the logits, which captures both the utility and intra-sample diversity of a sample, and an *inter-sample importance score* using low-dimensional similarity matching against previously selected samples to quantify inter-sample diversity (D1). Integrating these scores allows the training process to balance exploitation of high-utility samples with exploration of less-visited regions in the data distribution, improving the effectiveness of online batch selection.

In summary, we present **UDS (Utility-Diversity Sampling)**, a new framework for online batch selection in SFT of LLMs. Our main contributions are:

1. To capture intra-sample value, we leverage the nuclear norm of the logits matrix, which naturally reflects *data utility* and *intra-sample diversity*, providing a principled criterion for assessing within-sample informativeness.
2. To estimate inter-sample diversity, we design a structured bilinear random projection of logits for compact embedding and use similarity matching with historical data points to reduce redundancy with negligible overhead.
3. Our UDS operates directly on forward-pass outputs without external resources, achieving faster convergence than full-dataset SFT and surpassing existing baselines.

2. Preliminaries

2.1. Problem Formulation of Online Batch Selection

At training iteration t , we draw a candidate batch $\mathcal{B}_t = \{(\mathbf{x}_t^i, y_t^i)\}_{i=1}^B$ from the corpus. The goal of online batch selection is to choose a subset $\hat{\mathcal{B}}_t \subseteq \mathcal{B}_t$ that maximizes the immediate (or near-term) benefit to the model. Let $s(\mathbf{x}_t^i; \boldsymbol{\theta}_t, \mathcal{H}_t) \in \mathbb{R}$ denote a per-sample importance score computed under the current model state $\boldsymbol{\theta}_t$ and optional history $\mathcal{H}_t$ (e.g., recent sample embeddings). The selection objective can be viewed as an optimization problem:

$$\hat{\mathcal{B}}_t = \arg \max_{S \subseteq \mathcal{B}_t, |S|=K} \sum_{i \in S} s(\mathbf{x}_t^i; \boldsymbol{\theta}_t, \mathcal{H}_t), \quad (1)$$

where $K \in \{1, \dots, B\}$ is the target batch size.

2.2. Autoregressive Language Modeling

Our work focuses on autoregressive language models that factorize the sequence probability via the chain rule:

$$p_{\theta_t}(x_t^i) = \prod_{n=1}^N p_{\theta_t}(x_t^{i,n} | x_t^{i,<n}), \quad (2)$$

where $x_t^{i,<n} = (x_t^{i,1}, \dots, x_t^{i,n-1})$ denotes the context prefix, and $\theta_t \in \mathbb{R}^p$ represents model parameters at iteration t . At each generation step n , model π_{θ_t} produces a logit vector $l_t^{i,n} \in \mathbb{R}^V$ over vocabulary $\mathcal{V}$, with $V = |\mathcal{V}|$ the vocabulary size. For a sequence of length N , these vectors $l_t^{i,1}, \dots, l_t^{i,N}$ collectively form the logits matrix $L(x_t^i; \theta_t) \in \mathbb{R}^{N \times V}$, where the n -th row corresponds to logits at position n . $L(x_t^i; \theta_t)$ captures the model's predictive distribution at each position. Applying the softmax function to each row of $L(x_t^i; \theta_t)$ yields the conditional probability distribution for the next token. The resulting probability matrix is denoted $P(x_t^i; \theta_t) \in \mathbb{R}^{N \times V}$, with entries

$$p_{\theta_t}(x_t^{i,n} | x_t^{i,<n}) = \frac{\exp(\pi_{\theta_t}(x_{<n})[x_n])}{\sum_{y \in \mathcal{V}} \exp(\pi_{\theta_t}(x_{<n})[y])}. \quad (3)$$

During fine-tuning, the parameters θ_t are updated to maximize the likelihood of training sequences. At iteration t , the online selection mechanism identifies the most valuable subset $\hat{\mathcal{B}}_t$ from the candidate batch $\mathcal{B}_t$, which is then used to perform a parameter update $\theta_t \rightarrow \theta_{t+1}$.

3. Utility-Diversity Sampling (UDS)

We propose **UDS**, an online batch selection framework that jointly considers data utility, intra-sample diversity, and inter-sample diversity. It scores and samples each data point using a mixture of two complementary scores: (i) the *Nuclear Norm* of the logits matrix ($s_{\text{intra}}^{t,i}$), capturing optimization utility and intra-sample diversity; and (ii) the *Diversity Distance* ($s_{\text{inter}}^{t,i}$), measuring dispersion against historical selections. An overview is shown in Figure 1, with pseudocode provided in Algorithm 1.

3.1. Intra-sample Importance Score via Nuclear Norm

The intra-sample importance score ($s_{\text{intra}}^{t,i}$) can be characterized through two complementary views: (i) *optimization utility*—how much the sample can contribute to loss reduction during training, and (ii) *intra-sample diversity*—how diverse the model's token-level outputs are within the sequence. Inspired by Cui et al. (2020), we employ the nuclear norm (trace norm) of the logits matrix $L(x_t^i; \theta_t)$ to capture both aspects. A strong correlation between the nuclear norm $\|L(x_t^i; \theta_t)\|_*$ and these two aspects is visualized in Figures 2a and 2b. $\|L(x_t^i; \theta_t)\|_*$ is defined as the sum of the singular values of $L(x_t^i; \theta_t)$. Let $\{\sigma_1, \sigma_2, \dots, \sigma_r\}$ denote its

singular values (with $r = \text{rank}(L(x_t^i; \theta_t))$), then $s_{\text{intra}}^{t,i}$ is computed as:

$$s_{\text{intra}}^{t,i} = \|L(x_t^i; \theta_t)\|_* = \sum_{j=1}^r \sigma_j. \quad (4)$$

In mathematics, the nuclear norm and the Frobenius norm are related via loose bounds, as shown in Lemma 3.1. A detailed explanation is provided in Appendix D.1.

Due to the significant gap between the lower and upper bounds in Lemma 3.1, a larger nuclear norm can arise in two ways: (1) the Frobenius norm increases, typically indicating larger logits, and (2) for a fixed Frobenius norm, the nuclear norm shifts closer to the upper bound. Below, we intuitively clarify how these two effects connect to the two complementary aspects of the sample-importance score.

Lemma 3.1. (Horn & Johnson, 2012) For any matrix $L(x_t^i; \theta_t) \in \mathbb{R}^{N \times V}$, the following inequality holds:

$$\|L(x_t^i; \theta_t)\|_F \leq \|L(x_t^i; \theta_t)\|_* \leq C \cdot \|L(x_t^i; \theta_t)\|_F,$$

where $C = \sqrt{\min(N, V)}$ and $\|\cdot\|_F$ denotes the Frobenius norm, computed as the square root of the sum of squares of all entries in the matrix. The left inequality holds with equality when $L(x_t^i; \theta_t)$ is rank-1 (i.e., has only one non-zero singular value). The right inequality holds with equality when $L(x_t^i; \theta_t)$ has full rank and all singular values are equal.

Larger Nuclear Norm Correlates With Higher Optimization Utility. Traditional notions of sample difficulty in online batch selection—such as *maximum loss* (Loshchilov & Hutter, 2015) or *maximum gradient* (Katharopoulos & Fleuret, 2018)—often misalign with actual training dynamics (Wang et al., 2024). Instead, we characterize sample importance through its potential contribution to loss reduction, which has been widely used in previous work and which we term *optimization utility*. For ease of exposition, we assume training with SGD using batch size B and a learning rate η_t . An extension to Adam is provided in Appendix D.2. The parameter update at iteration t is $\theta_{t+1} - \theta_t = -\frac{\eta_t}{B} \sum_{j=1}^B \nabla_{\theta} \ell(x_t^j; \theta_t)$. After the update, the logits matrix for datapoint x_t^i can be expanded using a first-order Taylor expansion:

$$L(x_t^i; \theta_{t+1}) \approx L(x_t^i; \theta_t) + \nabla_{\theta} L(x_t^i; \theta_t) \cdot (\theta_{t+1} - \theta_t), \quad (5)$$

so that the induced change is

$$\begin{aligned} \delta L(x_t^i; \theta_t) &= L(x_t^i; \theta_{t+1}) - L(x_t^i; \theta_t) \\ &\approx -\frac{\eta_t}{B} \nabla_{\theta} L(x_t^i; \theta_t) \cdot \left(\sum_{j=1}^B \nabla_{\theta} \ell(x_t^j; \theta_t) \right). \end{aligned} \quad (6)$$

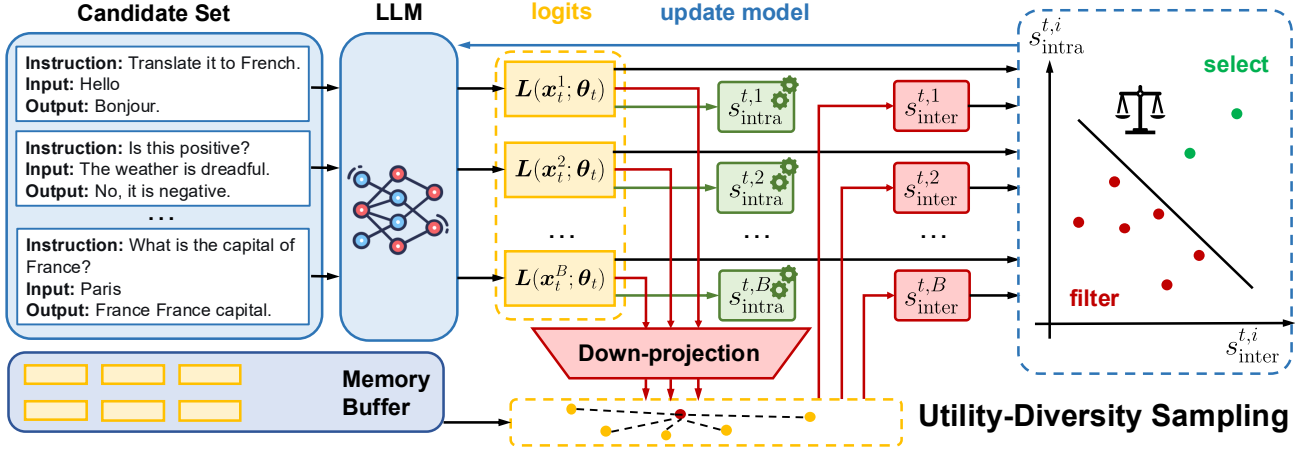


Figure 1. **Schematic of the UDS Framework.** In the forward pass, the LLM extract logits $L(x_i^j; \theta_t)$ for each sample. Using $L(x_i^j; \theta_t)$, we first calculate its nuclear norm for the intra-sample importance score $s_{intra}^{t,i}$, and then down-projected to calculate the distance $s_{inter}^{t,i}$ with historical samples in the memory buffer. Finally, we select the top- K valuable samples to update the LLM.

Algorithm 1 Utility-Diversity Sampling (UDS)

Input: Candidate batch sequence $\mathcal{B}_t$ for $t = 0, \dots, T$;
Initial model π_{θ_0} ; Memory queue capacity M ; Can-
didate batch size B ; Target batch size K ; Trade-off
factor α .

Output: Finetuned Model $\pi_{\theta_{T+1}}$

Initialize memory queue $Q \leftarrow \emptyset$

for $t = 0$ **to** T **do**

 // Calculate Importance Score

foreach $x_i^j \in \mathcal{B}_t$ **do**

$L(x_i^j; \theta_t) \leftarrow \pi_{\theta_t}(x_i^j)$

$s_{intra}^{t,i} \leftarrow \|L(x_i^j; \theta_t)\|_*$

$z_t^i \leftarrow \text{vec}(\Gamma_2 \cdot L(x_i^j; \theta_t) \cdot \Gamma_1^\top)$

$s_{inter}^{t,i} \leftarrow \frac{1}{|Q|} \sum_{z_j \in Q} \|z_t^i - z_j\|_2$

$s_{total}^{t,i} \leftarrow s_{intra}^{t,i} + \alpha \cdot s_{inter}^{t,i}$

 // Select top- K samples

$\hat{\mathcal{B}}_t \leftarrow \text{TopK}(\{s_{total}^{t,i}\}_{i=1}^B, K)$

 // Update Memory and Model

while $|Q| + K > M$ **do**

$Q \leftarrow Q \setminus \{\text{oldest element}\}$

$Q \leftarrow Q \cup \{z_t^i \mid x_i^j \in \hat{\mathcal{B}}_t\}$

 Update parameters θ_t through backpropagation on $\hat{\mathcal{B}}_t$

We can view the training step as producing a logits perturbation $\delta L(x_i^j; \theta_t)$. The first-order Taylor expansion of the loss around $L(x_i^j; \theta_t)$ gives

$$\begin{aligned} \delta \ell(x_i^j; \theta_t) &:= \ell(L(x_i^j; \theta_t) + \delta L(x_i^j; \theta_t)) - \ell(L(x_i^j; \theta_t)) \\ &\approx \langle \nabla_L \ell(L(x_i^j; \theta_t)), \delta L(x_i^j; \theta_t) \rangle, \end{aligned} \quad (7)$$

where $\langle \cdot, \cdot \rangle$ denotes the Frobenius inner product, and $\nabla_L \ell(L(x_i^j; \theta_t))$ is the gradient matrix of ℓ with respect to L . For cross-entropy loss, $\nabla_L \ell(L(x_i^j; \theta_t)) = P(x_i^j; \theta_t) -$

$Y(x_i^j)$, which we denote as $\Delta(x_i^j; \theta_t)$. Hence,

$$\delta \ell(x_i^j; \theta_t) \approx \langle \Delta(x_i^j; \theta_t), \delta L(x_i^j; \theta_t) \rangle. \quad (8)$$

Due to model complexity and nonlinearity, strictly proving the nuclear norm's linear correlation with loss reduction is challenging. Instead, we provide an intuitive explanation. According to Equation 8, as the Frobenius norm of the logits $\|L(x_i^j; \theta_t)\|_F$ increases, the perturbation $\delta L(x_i^j; \theta_t)$ also tends to grow uniformly. This occurs because each entry in the activation logits becomes larger, and these values are directly used during backpropagation. In contrast, $\Delta(x_i^j; \theta_t)$ is less sensitive to scale because it depends only on the normalized probabilities and the one-hot labels. Therefore, $\|L(x_i^j; \theta_t)\|_F$, $\|L(x_i^j; \theta_t)\|_*$, and the attainable loss reduction $-\delta \ell(x_i^j; \theta_t)$ typically grow in tandem. Empirically, Figure 2a and Figure 2c confirm a strong correlation between a sample's loss reduction and its nuclear norm and Frobenius norm, respectively. Hence, the nuclear norm can serve as an indicator of a sample's optimization utility.

When a single sample induces a larger loss reduction, it indicates that the sample both challenges the model's current predictions and provides strong gradients that effectively guide parameter updates. Such samples are particularly valuable for data selection: they not only highlight under-learned regions of the input space, but also accelerate training by yielding stronger optimization signals compared to redundant or already well-learned samples. We provide a categorization of the loss before and after training in Table 2. We also provide a discussion from a curriculum learning perspective in Appendix B.2 to analyze what kinds of data are selected as the training evolves. Hence, we prioritize selecting samples with larger nuclear norm that are more likely to have higher loss reduction.

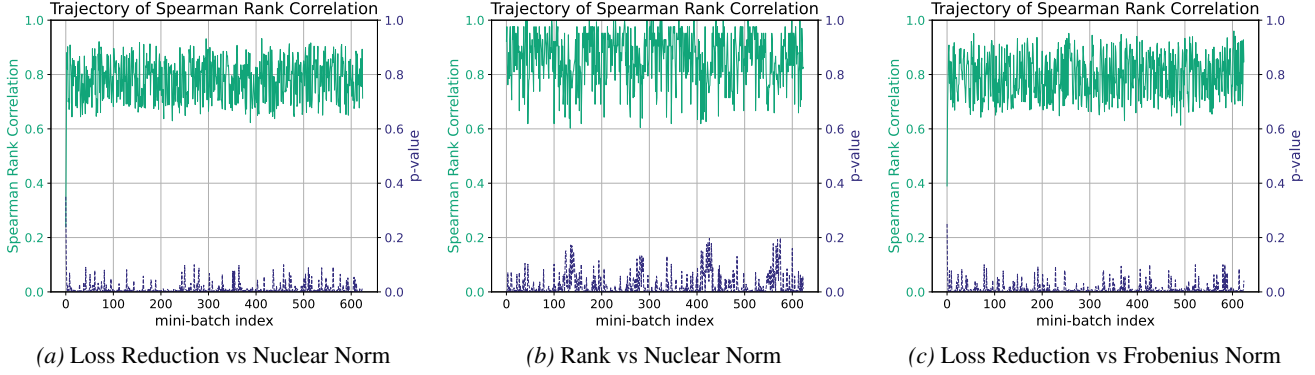


Figure 2. **Correlation Analysis using Qwen-2.5-7B on MMLU.** (a) Strong linear correlation between $-\delta\ell(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ and $\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_*$. (b) Strong linear correlation between $\text{rank}(\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t))$ and $\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_*$. (c) Strong linear correlation between $-\delta\ell(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ and $\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_F$. During training optimized with Adam, each point represents the correlation coefficient computed from B pairs per batch.

Table 2. **Categorization of training samples by initial and final loss.** The optimization utility depends on how much the training loss decreases after training. **Green** entries denote high-utility samples that should be preferentially selected, while **Red** entries denote low-utility ones that should generally be avoided.

Before/After	High	Low
High	Too Hard	High Utility
Low	Overfitted	Too Easy

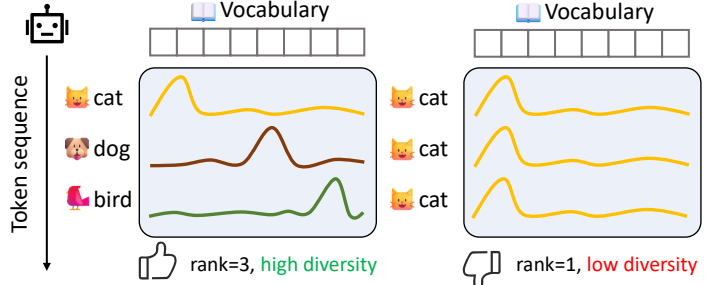


Figure 3. **Illustration of Intra-sample Diversity.** **Left:** Token sequence with high diversity, where the model predicts varied tokens (cat, dog, bird). **Right:** Token sequence with low diversity, where the model predominantly predicts a single token (cat).

Larger Nuclear Norm Correlates With Higher Intra-sample Diversity. Beyond optimization utility, a larger nuclear norm can also arise from the structural diversity of token-level output distributions within a sequence. Recall Lemma 3.1: for a fixed Frobenius norm, the nuclear norm achieves its *minimum* when $\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ is rank-1 with only one singular value, and its *maximum* when $\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ has full rank with equal singular values. These two extremes correspond to distinct regimes of intra-sample diversity. The lower bound typically occurs when the model predicts only a single vocabulary token throughout the sequence (indicating repetitive generation behavior), while the upper bound is achieved when the model produces a florid and diverse vocabulary distribution across tokens (reflecting richer semantic information content). Figure 3 presents a simplified example illustrating what intra-sample diversity means in real cases. Intuitively, A larger nuclear norm implies that more vocabularies in this sequence are predicted and utilized, indicating higher prediction dispersity. Figure 2b further supports this through the strong empirical correlation between $\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_*$ and $\text{rank}(\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t))$.

(i) Low Nuclear Norm Correlates with Low rank and

collinear rows (low diversity). Considering the lower bound of Lemma 3.1, rank-1 $\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ implies near collinear row vectors $\mathbf{l}_n \approx \alpha_n \mathbf{v}$ for some fixed $\mathbf{v}$. This degenerate spectral structure indicates minimal diversity: the model’s predictions for different tokens are aligned along the same direction, resulting in repetitive output.

(ii) High Nuclear Norm Correlates with High rank and orthogonal rows (high diversity). Considering the upper bound of Lemma 3.1, $\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ has full rank and equal singular values. This occurs when the rows are diverse and orthogonal with comparable norms. This flat spectrum indicates maximal diversity: each token is semantically meaningful, and predictions are spread across diverse directions, capturing a wide range of semantic information.

3.2. Inter-Sample Importance Score via Low-rank Similarity Matching

Current online batch selection methods (Wang et al., 2024) only consider diversity within the candidate batch, which is suboptimal because the capacity of the batch is much smaller than the global datasets. To enhance global diversity, we maintain a fixed-size First-In-First-Out (FIFO) memory

buffer $Q \in \mathbb{R}^{M \times d}$, which stores representations $z_t^i \in \mathbb{R}^d$ of the last M samples selected for training ($M \gg B$), and measure inter-sample diversity by calculating the distance between the candidate sample and the recent training history.

Diversity Distance: The inter-sample diversity score for the i -th sample is computed as its average Euclidean distance to all representations in the memory buffer:

$$s_{\text{inter}}^{t,i} = \underbrace{\frac{1}{|Q|} \sum_{z_j \in Q} \|z_t^i - z_j\|_2}_{\text{diversity w.r.t. buffer}} + \underbrace{\frac{1}{|B_t|} \sum_{z_k \in B_t} \|z_t^i - z_k\|_2}_{\text{(optional) intra-batch diversity}} \quad (9)$$

If Q is empty, $s_{\text{inter}}^{t,i}$ is set to zero. A high $s_{\text{inter}}^{t,i}$ indicates a large distinction from recent data.

Remark: While within-batch diversity could theoretically improve performance, it is likely negligible since the batch size $B \ll M$ and data are shuffled. Similar samples rarely appear in a batch without being in the buffer. We thus omit this for simplicity. We provide an additional experiment in Appendix C.7 to validate this assumption.

Low-dimensional Projection. As mentioned before, the logits matrix $L(x_t^i; \theta_t) \in \mathbb{R}^{N \times V}$ is semantically meaningful for computing diversity, but directly storing the whole matrix in the memory buffer is prohibitive (e.g., $\sim 74\text{GB}$ for 1024 samples in Qwen-2.5-7B). A natural strategy is to randomly project it into low-dimensional vectors $z_t^i \in \mathbb{R}^d$, while preserving pairwise distances (Johnson et al., 1984). However, a direct projection using $\Gamma \in \mathbb{R}^{NV \times d}$ also incurs severe storage cost for the down-projection matrix ($\sim 74\text{GB}$ in Qwen-2.5-7B if d is set to 1024). To avoid this, we try to factorize Γ into two smaller projections: $\Gamma_1 \in \mathbb{R}^{d_1 \times V}$ reducing the vocabulary dimension and $\Gamma_2 \in \mathbb{R}^{d_2 \times N}$ reducing the sequence length dimension, which together approximate the original projection with $d = d_1 d_2$. Formally, we aim to find proper Γ_1 and Γ_2 that operate in the following way:

$$z_t^i = \text{vec}(\Gamma_2 \cdot L(x_t^i; \theta_t) \cdot \Gamma_1^\top), \quad (10)$$

Fortunately, the *subsampled randomized Fourier transform* (SRFT)-style construction for Γ_1 and Γ_2 (see Theorem 3.2) can fulfill this role (Ailon & Chazelle, 2006; Jin et al., 2021). This approach achieves an approximate Johnson-Lindenstrauss embedding while avoiding the storage of an explicit $NV \times d$ projection matrix and reduces the computational complexity from $\mathcal{O}(NVd)$ to $\mathcal{O}((N+V)d \log(NV))$. The complete proof is provided in Appendix D.3.

Theorem 3.2. Let $N, V \in \mathbb{N}$, and choose $d_1 \leq V$, $d_2 \leq N$ with $d = d_1 d_2$. Define $\Gamma_1 \in \mathbb{R}^{d_1 \times V}$ and $\Gamma_2 \in \mathbb{R}^{d_2 \times N}$ as

$$\Gamma_1 = \sqrt{\frac{V}{d_1}} S_1 F_1 D_1, \quad \Gamma_2 = \sqrt{\frac{N}{d_2}} S_2 F_2 D_2,$$

where F_1, F_2 are orthonormal transforms (discrete Fourier transform (DFT) matrices), D_1, D_2 are random $\{\pm 1\}$ diagonal matrices with independent Rademacher entries on the diagonal, and S_1, S_2 are random selection matrices that choose d_1 and d_2 rows uniformly at random without replacement. For each $L(x_t^i; \theta_t) \in \mathbb{R}^{N \times V}$, define $u_t^i \in \mathbb{R}^{NV}$ and $v_t^i \in \mathbb{R}^d$

$$u_t^i = \text{vec}(L(x_t^i; \theta_t)), \quad v_t^i = \text{vec}(\Gamma_2 \cdot L(x_t^i; \theta_t) \cdot \Gamma_1^\top).$$

Then there exists a constant $C > 0$ such that for any ϵ and a finite set of n points, if $d \geq C\epsilon^{-2} \log(NV) \text{polylog}(n)$, the mapping $u_t^i \mapsto v_t^i$ satisfies the Johnson-Lindenstrauss lemma with high probability. Then, for all i, j ,

$$(1-\epsilon)\|u_t^i - u_t^j\|_2^2 \leq \|v_t^i - v_t^j\|_2^2 \leq (1+\epsilon)\|u_t^i - u_t^j\|_2^2.$$

3.3. Selection Criterion

After computing the intra- and inter-sample importance scores $s_{\text{intra}}^{t,i}$ and $s_{\text{inter}}^{t,i}$, we combine them to obtain a joint score for each sample:

$$s_{\text{total}}^{t,i} = s_{\text{intra}}^{t,i} + \alpha s_{\text{inter}}^{t,i}, \quad (11)$$

where α is a trade-off factor. Based on $s_{\text{total}}^{t,i}$, we then select the top- K samples from the current batch B_t for training:

$$\hat{B}_t = \arg \max_{S \subseteq B_t, |S|=K} \sum_{i \in S} s_{\text{total}}^{t,i}. \quad (12)$$

The resulting subset selection module is plug-and-play, directly integrating with the SFT pipeline to enhance its performance while maintaining efficiency.

4. Experiments

4.1. Experimental Setup

Datasets and Backbones: We mainly evaluate online batch selection methods' performance across four key domains: (1) *general knowledge understanding* using the MMLU (Hendrycks et al., 2021a) benchmark, with auxiliary training datasets for fine-tuning and the official test set for evaluation; (2) *scientific question answering* using ScienceQA (Lu et al., 2022) for both fine-tuning and evaluation; (3) *mathematical reasoning* on GSM8K (Cobbe et al., 2021) for both fine-tuning and evaluation; and (4) *code generation*

Table 3. Performance Comparison for Different Online Batch Selection Methods. We evaluate various methods across four benchmarks: MMLU, ScienceQA, GSM8K, and HumanEval. $\bar{A}$ denotes average accuracy or Pass@1 reported in percentage (%). $\mathcal{T}_{\text{train}}$ denotes throughput during the training time (samples per second). The best results of $\bar{A}$ are highlighted in **bold**. Methods training faster than the full dataset are marked in , others are marked in .

Model	Method	MMLU		ScienceQA		GSM8K		HumanEval	
		$\bar{A}(\uparrow)$	$\mathcal{T}_{\text{train}}(\uparrow)$	$\bar{A}(\uparrow)$	$\mathcal{T}_{\text{train}}(\uparrow)$	$\bar{A}(\uparrow)$	$\mathcal{T}_{\text{train}}(\uparrow)$	$\bar{A}(\uparrow)$	$\mathcal{T}_{\text{train}}(\uparrow)$
Llama-3.1-8B	Regular	38.24 $\pm$ 0.35	2.09	93.19 $\pm$ 0.65	6.61	55.95 $\pm$ 0.47	3.73	29.28 $\pm$ 0.48	5.77
	Random	35.47 $\pm$ 1.25	3.74	92.86 $\pm$ 0.27	10.06	54.89 $\pm$ 0.73	6.68	26.74 $\pm$ 0.56	8.92
	MaxLoss	35.62 $\pm$ 0.79	2.75	92.82 $\pm$ 0.21	7.49	55.42 $\pm$ 0.35	4.98	27.23 $\pm$ 0.27	6.62
	MaxGrad	35.91 $\pm$ 1.17	0.29	92.77 $\pm$ 0.24	0.94	55.08 $\pm$ 0.58	0.53	26.89 $\pm$ 0.74	0.80
	RHO-Loss	37.63 $\pm$ 0.67	1.40	93.42 $\pm$ 0.15	3.83	56.54 $\pm$ 0.52	2.53	27.18 $\pm$ 0.29	3.36
	GREATS	39.04 $\pm$ 0.29	1.88	93.68 $\pm$ 0.19	6.04	57.03 $\pm$ 0.33	3.37	28.56 $\pm$ 0.34	5.25
	UDS (Ours)	40.16$\pm$0.58	2.48	94.33$\pm$0.28	7.46	58.98$\pm$0.24	4.59	30.96$\pm$0.69	6.41
Qwen-2.5-7B	Regular	55.32 $\pm$ 0.79	2.27	94.56 $\pm$ 0.17	7.05	78.23 $\pm$ 0.07	3.77	45.82 $\pm$ 0.41	6.24
	Random	54.26 $\pm$ 1.85	9.29	93.28 $\pm$ 0.35	10.27	77.69 $\pm$ 0.29	9.18	40.20 $\pm$ 1.18	9.35
	MaxLoss	54.51 $\pm$ 1.37	5.93	93.05 $\pm$ 0.40	7.93	77.78 $\pm$ 0.36	5.45	41.34 $\pm$ 0.83	6.92
	MaxGrad	54.33 $\pm$ 0.69	0.31	93.86 $\pm$ 0.51	0.98	77.62 $\pm$ 0.28	0.53	40.83 $\pm$ 0.41	0.88
	RHO-Loss	57.08 $\pm$ 0.74	1.94	93.80 $\pm$ 0.25	3.89	78.38 $\pm$ 0.43	3.08	43.08 $\pm$ 1.62	3.53
	GREATS	58.19 $\pm$ 0.49	2.12	94.17 $\pm$ 0.62	6.53	78.61 $\pm$ 0.41	3.50	45.04 $\pm$ 0.59	5.80
	UDS (Ours)	63.34$\pm$0.36	3.41	95.19$\pm$0.22	7.85	79.91$\pm$0.23	4.99	46.28$\pm$0.35	6.81

Table 4. Ablation study of UDS components across multiple benchmarks using Qwen-2.5-7B. $\bar{A}$ denotes average accuracy or Pass@1 reported in percentage (%). We report $\bar{A}$ and relative improvement Δ over the baseline.

Method	MMLU		ScienceQA		GSM8K		HumanEval	
	$\bar{A}$ (%)	Δ	$\bar{A}$ (%)	Δ	$\bar{A}$ (%)	Δ	$\bar{A}$ (%)	Δ
Random (Baseline)	54.26 $\pm$ 1.85	—	93.28 $\pm$ 0.35	—	77.69 $\pm$ 0.29	—	40.20 $\pm$ 1.18	—
Only Nuclear Norm	58.35 $\pm$ 0.76	+4.09	94.19 $\pm$ 0.31	+0.91	79.22 $\pm$ 0.16	+1.53	44.18 $\pm$ 0.55	+3.98
Only Diversity Distance	57.75 $\pm$ 1.48	+3.49	93.98 $\pm$ 0.24	+0.70	78.96 $\pm$ 0.31	+0.67	43.84 $\pm$ 0.19	+3.64
UDS (Full)	63.34$\pm$0.36	+9.08	95.19$\pm$0.22	+1.91	79.91$\pm$0.23	+2.22	46.28$\pm$0.35	+6.08

using CodeAlpaca-20k (Chaudhary, 2023) for training and HumanEval (Chen et al., 2021) for evaluation. All experiments are conducted using Llama-3.1-8B (Grattafiori et al., 2024) and Qwen-2.5-7B (Yang et al., 2024) as backbone models following (Zou et al., 2025c).

Baselines: We mainly compare UDS against the following online batch selection baselines: (1) **Regular**, where all samples are used without data selection; (2) **Random Selection**, which randomly chooses samples from batches; (3) **MaxLoss** (Loshchilov & Hutter, 2015), which prioritizes samples with the highest training loss; (4) **MaxGrad** (Katharopoulos & Fleuret, 2018), which selects samples with the largest gradient norms; (5) **RHO-Loss** (Mindermann et al., 2022), a representative method using a reference model; and (6) **GREATS** (Wang et al., 2024), a state-of-the-art (SOTA) online batch selection approach. All baselines select the same fraction of data for a fair comparison.

Implementation Details: Under hardware constraints, we employ LoRA (rank=8) for SFT training. The batch size

is set to $B = 8$ for all datasets. The optimal data selection ratio α is highly dependent on both the backbone model and the dataset; the ratios we adopt for different combinations of backbones and datasets are reported in Table 6. The default choice of the buffer size $M = 1024$, the down-projection dimension is $d_1 = 128$, $d_2 = 8$, and we use this across all the experiments. We report average accuracy on MMLU, ScienceQA, and GSM8K, and Pass@1 for HumanEval. We also compare training speed by throughput relative to an NVIDIA GeForce RTX 3090 GPU, following Wang et al. (2024). All benchmarks use zero-shot evaluation and are repeated four times with different random seeds. More detailed experimental settings are provided in Appendix E.

4.2. Highest Accuracy with Lower Training Time than Full Dataset

UDS achieves the highest accuracy. Across all four benchmarks, UDS consistently delivers the best accuracy among online batch selection methods. On MMLU, it

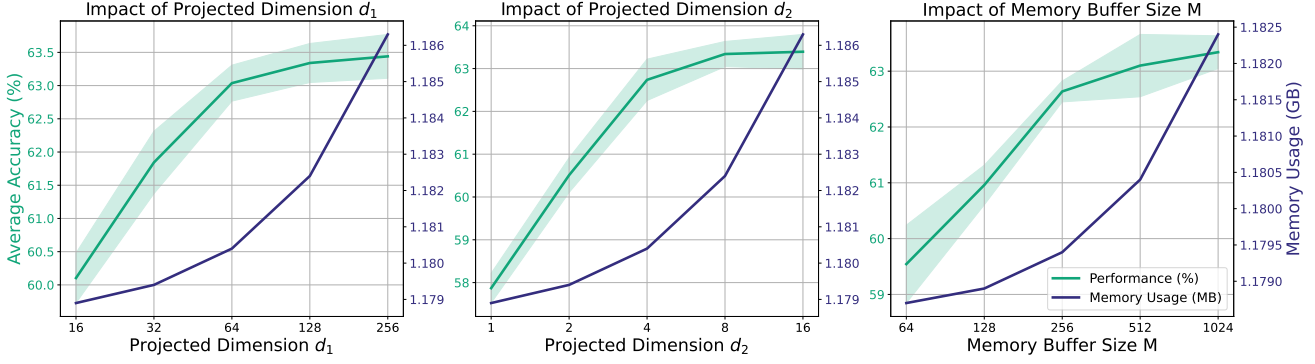


Figure 4. Impact of d_1 , d_2 , and M on Qwen-2.5 performance and memory usage on MMLU. Green curves show average accuracy; blue curves show peak memory. Performance improves with increasing d_1 , d_2 , and M , while memory usage increases only slightly.

achieves 63.34%, outperforming GREATS by +5.15% when using Qwen-2.5-7B. Similar gains appear on ScienceQA (95.19% vs. 94.17%), GSM8K (79.91% vs. 78.61%), and HumanEval (46.28% vs. 45.04%). The improvements hold across both Llama-3.1-8B and Qwen-2.5-7B, showing strong generalization. Overall, UDS surpasses simple heuristics (MaxLoss, MaxGrad) and advanced baselines (RHO-Loss, GREATS), achieving SOTA performance in the online batch selection setting.

UDS is more efficient than training on the full dataset. Beyond accuracy, UDS maintains competitive efficiency relative to training on the full dataset. On Qwen-2.5-7B, it achieves 3.41 samples/s on MMLU and 6.81 on HumanEval, both higher than those of full-dataset training (2.27 and 6.24). On Llama-3.1-8B, it also sustains higher throughput while yielding better accuracy. MaxLoss also trains quickly but provides only marginal accuracy gains, whereas MaxGrad slows training dramatically with no significant benefit. GREATS, though accurate, consistently runs slower than UDS. Thus, UDS achieves the best trade-off, combining high accuracy with efficiency.

Appendices C.3–C.5 provide further evaluations on full SFT, larger batch sizes, instruction-tuning models, Long-CoT, and OOD datasets, validating the robustness of UDS.

4.3. Ablation Studies

Both components of UDS contribute to performance. Table 4 summarizes the ablations on the two components of UDS. *Nuclear Norm* alone consistently outperforms random selection, underscoring the importance of capturing intra-sample utility and diversity. Similarly, *Diversity Distance* improves upon the baseline by reducing redundancy. Combined, the two components complement each other to achieve the best results across all benchmarks, highlighting the necessity of jointly modeling sample utility, intra-sample diversity, and inter-sample diversity.

Appendix C.2 provides additional ablations on buffer policies, distance aggregation, and random matrix constructions.

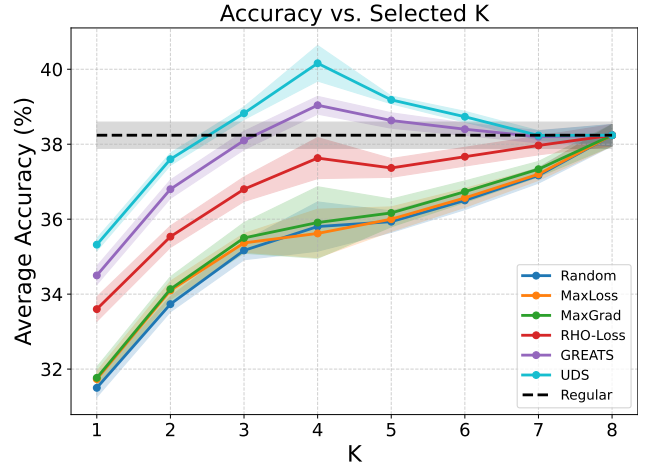


Figure 5. Performance across different data scales using Llama-3.1-8B on MMLU. We report accuracy when fine-tuning with varying proportions of training data. While all methods improve with more data, UDS consistently achieves the best accuracy and surpasses full-dataset fine-tuning.

4.4. Hyperparameter Analysis

Figure 4 shows the key design of the bilinear down-projection and memory buffer incur no severe memory burden. Appendix C.1 further analyzes the impact of M , d_1 , d_2 regarding accuracy and resources, and α regarding accuracy.

4.5. Comparison across Different Data Scales

To assess the effectiveness of UDS under varying batch selection fractions, we conduct experiments as shown in Figure 5. We report the number of selected samples K per batch versus average accuracy across baselines.

While most methods improve with K , UDS’s performance peaks at $K=4$, achieving the highest accuracy before gradually declines as additional, less valuable samples are added. When $K=B=8$, all methods reduce to full-dataset fine-tuning. UDS consistently outperforms baselines, showing its ability to prioritize valuable and diverse samples. At its peak, UDS even surpasses full-dataset fine-tuning, demonstrating that a small, well-curated subset can deliver both

efficiency and accuracy. These results confirm that UDS effectively balances exploitation and exploration, yielding robust improvements under different selection budgets.

5. Conclusion

In this work, we systematically analyze the limitations of current online batch selection methods for supervised fine-tuning of LLMs and establish three fundamental desiderata for optimal selection strategy design. To fulfill these requirements, we propose UDS, a novel framework that synergistically combines two complementary components: (i) Nuclear Norm scoring to capture both optimization utility and intra-sample diversity, and (ii) Diversity Distance measurement to ensure inter-sample diversity through efficient historical comparison. Extensive experiments across multiple domains illustrate UDS’s superior performance over existing methods while maintaining computational efficiency. Our approach provides a competitive and practical solution for effective online batch selection and holds the potential of scaling to more efficient LLM training scenarios.

Acknowledgment

Dr. Qi Wang acknowledges support from the National Natural Science Foundation of China (NSFC) under the grant number 62306326. This work was supported by the National Science and Technology Major Project 2025ZD1606303, Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China JYB2025XD503 and National Natural Science Foundation of China 62495092.

Impact Statement

This work presents an algorithmic advancement intended to streamline the SFT process through efficient online batch selection. By enabling models to learn from high-utility, diverse samples without the overhead of full-dataset training or external reference models, we provide a pathway to optimize resource utilization in machine learning workflows. As a general-purpose optimization technique, the proposed framework operates within the boundaries of existing datasets and architectures; thus, it does not generate novel societal risks. The broader impact of the technology will be determined by the specific downstream tasks and data distributions applied by practitioners.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Ailon, N. and Chazelle, B. Approximate nearest neighbors and the fast johnson-lindenstrauss transform. In *Proceedings of the thirty-eighth annual ACM symposium on Theory of computing*, pp. 557–563, 2006.
- Ailon, N. and Chazelle, B. The fast johnson–lindenstrauss transform and approximate nearest neighbors. *SIAM Journal on computing*, 39(1):302–322, 2009.
- Albalak, A., Elazar, Y., Xie, S. M., Longpre, S., Lambert, N., Wang, X., Muennighoff, N., Hou, B., Pan, L., Jeong, H., et al. A survey on data selection for language models. *arXiv preprint arXiv:2402.16827*, 2024.
- Axiotis, K., Cohen-Addad, V., Henzinger, M., Jerome, S., Mirrokni, V., Saulpic, D., Woodruff, D., and Wunder, M. Data-efficient learning via clustering-based sensitivity sampling: Foundation models and beyond. *arXiv preprint arXiv:2402.17327*, 2024.
- Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chaudhary, S. Code alpaca: An instruction-following llama model for code generation. <https://github.com/sahil280114/codealpaca>, 2023.
- Chen, L., Li, S., Yan, J., Wang, H., Gunaratna, K., Yadav, V., Tang, Z., Srinivasan, V., Zhou, T., Huang, H., et al. Alpapasus: Training a better alpaca with fewer data. *arXiv preprint arXiv:2307.08701*, 2023.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code, 2021.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano,

- R., Hesse, C., and Schulman, J. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Cui, S., Wang, S., Zhuo, J., Li, L., Huang, Q., and Tian, Q. Towards discriminability and diversity: Batch nuclear-norm maximization under label insufficient situations. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 3941–3950, 2020.
- Deb, R., Thekumparampil, K., Kalantari, K., Hiranandani, G., Sabach, S., and Kveton, B. Fishersft: Data-efficient supervised fine-tuning of language models using information gain. *arXiv preprint arXiv:2505.14826*, 2025.
- Deng, Z., Cui, P., and Zhu, J. Towards accelerated model training via bayesian data selection. *Advances in Neural Information Processing Systems*, 36:8513–8527, 2023.
- Geng, J., Cai, F., Wang, Y., Koepl, H., Nakov, P., and Gurevych, I. A survey of confidence estimation and calibration in large language models. *arXiv preprint arXiv:2311.08298*, 2023.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021a.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021b.
- Horn, R. A. and Johnson, C. R. *Matrix analysis*. Cambridge university press, 2012.
- Huang, Y., Song, J., Wang, Z., Zhao, S., Chen, H., Juefei-Xu, F., and Ma, L. Look before you leap: An exploratory study of uncertainty measurement for large language models. *arXiv preprint arXiv:2307.10236*, 2023.
- Jiang, A. H., Wong, D. L.-K., Zhou, G., Andersen, D. G., Dean, J., Ganger, G. R., Joshi, G., Kaminsky, M., Kozuch, M., Lipton, Z. C., et al. Accelerating deep learning by focusing on the biggest losers. *arXiv preprint arXiv:1910.00762*, 2019.
- Jin, R., Kolda, T. G., and Ward, R. Faster johnson-lindenstrauss transforms via kronecker products. *Information and Inference: A Journal of the IMA*, 10(4):1533–1562, 2021.
- Johnson, W. B., Lindenstrauss, J., et al. Extensions of lipschitz mappings into a hilbert space. *Contemporary mathematics*, 26(189-206):1, 1984.
- Kaddour, J., Key, O., Nawrot, P., Minervini, P., and Kusner, M. J. No train no gain: Revisiting efficient training algorithms for transformer-based language models. *Advances in Neural Information Processing Systems*, 36:25793–25818, 2023.
- Katharopoulos, A. and Fleuret, F. Not all samples are created equal: Deep learning with importance sampling. In *International conference on machine learning*, pp. 2525–2534. PMLR, 2018.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., and Carlini, N. Deduplicating training data makes language models better. *arXiv preprint arXiv:2107.06499*, 2021.
- Li, D., Ma, Y., Wang, N., Ye, Z., Cheng, Z., Tang, Y., Zhang, Y., Duan, L., Zuo, J., Yang, C., et al. Mixlora: Enhancing large language models fine-tuning with lora-based mixture of experts. *arXiv preprint arXiv:2404.15159*, 2024.
- Li, R., Zou, H., Yan, X., Liang, Z., Yang, J., Li, C., and Yang, X. Enhancing pretrained model-based continual representation learning via guided random projection. *arXiv preprint arXiv:2603.19145*, 2026.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Loshchilov, I. and Hutter, F. Online batch selection for faster training of neural networks. *arXiv preprint arXiv:1511.06343*, 2015.
- Lu, P., Mishra, S., Xia, T., Qiu, L., Chang, K.-W., Zhu, S.-C., Tafjord, O., Clark, P., and Kalyan, A. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *The 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
- Mao, Y., Qu, Y., Wang, Q., Zou, H., and Ji, X. Dynamics-predictive sampling for active rl finetuning of large reasoning models. *arXiv preprint arXiv:2603.10887*, 2026.

- Mindermann, S., Brauner, J. M., Razzak, M. T., Sharma, M., Kirsch, A., Xu, W., Höltgen, B., Gomez, A. N., Morisot, A., Farquhar, S., et al. Prioritized training on points that are learnable, worth learning, and not yet learnt. In *International Conference on Machine Learning*, pp. 15630–15649. PMLR, 2022.
- Peng, R., Zou, H., Wang, H., Zeng, Y., Huang, Z., and Zhao, J. Energy-based automated model evaluation. *arXiv preprint arXiv:2401.12689*, 2024.
- Qiu, X. and Mäkeläinen, R. Semantic density: Uncertainty quantification for large language models through confidence measurement in semantic space. *Advances in neural information processing systems*, 37:134507–134533, 2024.
- Qu, Y., Wang, Q., Mao, Y., Hu, V. T., Ommer, B., and Ji, X. Can prompt difficulty be online predicted for accelerating rl finetuning of reasoning models? In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*, pp. 1240–1250, 2026a.
- Qu, Y., Wang, Q., Mao, Y., Zou, H., Jiang, Y., Liu, W., Bai, C., Yang, K., Chen, Y., Yang, S., et al. Small generalizable prompt predictive models can steer efficient rl post-training of large reasoning models. *arXiv preprint arXiv:2602.01970*, 2026b.
- Sachdeva, N., Coleman, B., Kang, W.-C., Ni, J., Hong, L., Chi, E. H., Caverlee, J., McAuley, J., and Cheng, D. Z. How to train data-efficient llms. *arXiv preprint arXiv:2402.09668*, 2024.
- Shorinwa, O., Mei, Z., Lidard, J., Ren, A. Z., and Majumdar, A. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *ACM Computing Surveys*, 2025.
- Sun, Y., Zhang, Q., Yu, Z., Zhang, X., Shen, L., and Tao, D. Maskpro: Linear-space probabilistic learning for strict (n:m)-sparsity on LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=0R06BghLJX>.
- Tirumala, K., Simig, D., Aghajanyan, A., and Morcos, A. D4: Improving llm pretraining via document deduplication and diversification. *Advances in Neural Information Processing Systems*, 36:53983–53995, 2023.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Tropp, J. A. Improved analysis of the subsampled randomized hadamard transform. *Advances in Adaptive Data Analysis*, 3(01n02):115–126, 2011.
- Wang, J. T., Wu, T., Song, D., Mittal, P., and Jia, R. Greats: Online selection of high-quality data for llm training in every iteration. *Advances in Neural Information Processing Systems*, 37:131197–131223, 2024.
- Wang, Q., Xiao, Z., Mao, Y., Qu, Y., Shen, J., Lv, Y., and Ji, X. Model predictive task sampling for efficient and robust adaptation. *arXiv preprint arXiv:2501.11039*, 2025.
- Wang, X., Chen, Y., and Zhu, W. A survey on curriculum learning. *IEEE transactions on pattern analysis and machine intelligence*, 44(9):4555–4576, 2021.
- Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N. A., Khashabi, D., and Hajishirzi, H. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.
- Wettig, A., Gupta, A., Malik, S., and Chen, D. Qurating: Selecting high-quality data for training language models. *arXiv preprint arXiv:2402.09739*, 2024.
- Xia, M., Malladi, S., Gururangan, S., Arora, S., and Chen, D. Less: Selecting influential data for targeted instruction tuning. *arXiv preprint arXiv:2402.04333*, 2024.
- Xiao, Z., Zhao, Y., Zhang, Q., Xie, J., Zhao, W., Zhang, W., Luo, X., Yu, P. S., and Zhang, M. Sample lottery: Unsupervised discovery of critical instances for LLM reasoning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=76OZBE4Rb6>.
- Xie, Q., Dai, Z., Hovy, E., Luong, T., and Le, Q. Unsupervised data augmentation for consistency training. *Advances in neural information processing systems*, 33: 6256–6268, 2020.
- Xie, S. M., Santurkar, S., Ma, T., and Liang, P. S. Data selection for language models via importance resampling. *Advances in Neural Information Processing Systems*, 36: 34201–34227, 2023.
- Xu, B., Zhang, L., Mao, Z., Wang, Q., Xie, H., and Zhang, Y. Curriculum learning for natural language understanding. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pp. 6095–6104, 2020.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Zhang, Q., Deng, Z., Chen, Z., Hu, H., and Yang, Y. Stochastic continuous submodular maximization: Boosting via non-oblivious function. In *International Conference on Machine Learning*, pp. 26116–26134. PMLR, 2022.

Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., et al. Lima: Less is more for alignment. *Advances in Neural Information Processing Systems*, 36:55006–55021, 2023.

Zou, H., Zang, Y., and Ji, X. Structural features of the fly olfactory circuit mitigate the stability-plasticity dilemma in continual learning. *arXiv preprint arXiv:2502.01427*, 2025a.

Zou, H., Zang, Y., Xu, W., and Ji, X. Fly-cl: A fly-inspired framework for enhancing efficient decorrelation and reduced training time in pre-trained model-based continual representation learning. *arXiv preprint arXiv:2510.16877*, 2025b.

Zou, H., Zang, Y., Xu, W., Zhu, Y., and Ji, X. FlyloRA: Boosting task decoupling and parameter efficiency via implicit rank-wise mixture-of-experts. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025c.

A. Related Work

Online Batch Selection for Large Language Models. Several studies have investigated online batch selection prior to the era of LLMs. Works like Jiang et al. (2019); Katharopoulos & Fleuret (2018); Loshchilov & Hutter (2015) propose heuristic methods that select “important” samples (e.g., via high training loss, or large gradient norm). These methods are straightforward and often effective for smaller models or simpler tasks, but more recent studies (Wang et al., 2024) show that these heuristics may not scale well for LLMs. Another paradigm uses a reference model to help evaluate sample importance. For instance, Deng et al. (2023); Mindermann et al. (2022) use a pre-trained or separately trained reference model, and select samples based on how “different” they are relative to that model. These methods tend to require extra computation (e.g. extra forward passes) and sometimes held-out data to train the reference model, which can make them costly or impractical in large-scale LLM training. (Kaddour et al., 2023) points out that these overheads can outweigh benefits in many settings. GREATS (Wang et al., 2024) is a more recent SOTA method. It selects data batches online by considering loss reduction on a validation set, and uses a “ghost inner-product” approximation (via a Taylor-expansion-based greedy algorithm) to accelerate selection. However, the need for a validation set may be unavailable or non-representative in practical deployment (Wang et al., 2024). Recent studies also extend online selection ideas to efficient RL fine-tuning of reasoning models (Wang et al., 2025; Qu et al., 2026a; Mao et al., 2026; Qu et al., 2026b).

Offline Data Selection for Large Language Models. Most existing approaches to data selection are performed in an offline manner, where samples are filtered prior to training (Albalak et al., 2024; Xia et al., 2024; Zhou et al., 2023; Chen et al., 2023; Xie et al., 2023; Wettig et al., 2024). This is primarily due to efficiency concerns, since curating the entire dataset at every iteration is prohibitively expensive. Early approaches extend uncertainty estimation and active learning heuristics, such as filtering by entropy or difficulty, but these remain limited in capturing truly informative samples (Xu et al., 2020). A more impactful direction is large-scale pruning and deduplication: (Lee et al., 2021; Tirumala et al., 2023) show that removing near-duplicate or low-quality data improves efficiency and prevents overfitting. In instruction tuning, several works highlight that “quality outweighs quantity”, with methods like LIMA (Zhou et al., 2023), LESS (Xia et al., 2024), and Self-Instruct (Wang et al., 2022) filtering diverse, representative subsets (Zhang et al., 2022) while discarding noisy prompts. Recent advances further explore offline data selection through various sampling strategies: Sachdeva et al. (2024) introduce LLM-as-judge method and density-based sampling; Axiotis et al. (2024) propose clustering-based sensitivity sampling; and Deb et al. (2025) leverage information gain estimation to identify high-value examples. While effective to some extent, this perspective assumes that the value of each example is fixed throughout the learning process. In reality, learning is highly dynamic. Data that appear highly informative at the beginning of training may later become redundant (Wang et al., 2024). This limitation motivates serving online batch selection as a complement for dynamically assessing sample value during the training process.

B. Discussions

B.1. Discussion on the Different Objectives of Offline Data Selection and Online Data Selection

Many data selection methods exist for SFT. However, most of them are conducted offline; that is, data selection is performed before model training begins. In this work, we focus on online batch selection for SFT. Offline and online selection methods serve different purposes and are not directly comparable, particularly in terms of training-time efficiency. Instead, they form a complementary pipeline: offline selection provides a coarse filtering stage to ensure dataset quality, while online selection adaptively captures the changing importance of samples throughout the training process, thereby improving overall efficiency and accuracy (Wang et al., 2024). In this sequential pipeline, the candidate data points for online batch selection are usually already carefully curated after offline data selection, from which “outliers, noisy data, or adversarial examples” have already been filtered. Nevertheless, even with a high-quality SFT dataset, the importance of samples can shift throughout the learning process (Wang et al., 2024). Compared to offline data selection, the core objective of online batch selection is to capture these dynamic changes in data importance during training, which is precisely what our UDS design addresses.

B.2. Discussion on the Role of Online Batch Selection from the Perspective of Curriculum Learning

The online batch selection setting can be viewed as a form of implicit curriculum learning, and many prior works on curriculum learning have shown the benefit of scheduling data in an “easy-to-hard” order (Wang et al., 2021). In this work, we define data utility as loss reduction. People may have the question of whether this criterion may prefer “easy” points for the model to learn quickly. Actually, if a data point can be learned sufficiently, that is indeed what we expect to select at this

stage. For other samples that are more complex, even if they are more informative, if the model is not yet strong enough to effectively capture their patterns (i.e., they are not learnable), they may not be useful for learning at the current stage. Within the implicit curriculum learning framework, as the model becomes stronger, it gains the ability to capture more complex data points; at that point, these more difficult samples can then be selected for training.

B.3. Discussion on the Difference between Online Batch Selection and Active Learning

While online batch selection and active learning both target data efficiency, they differ in their core assumptions and objectives. Classical active learning typically relies on a relatively static model snapshot to select samples, and thus struggles to capture the highly dynamic nature of training: data points that are informative early on may quickly become redundant as the model evolves. In contrast, online batch selection is embedded directly into the optimization loop and explicitly conditions selection on the current training state, allowing sample importance to change over time.

A second key distinction lies in the selection criteria. Active learning methods are predominantly driven by uncertainty estimation, which is well aligned with improving predictive performance (Peng et al., 2024) but only indirectly related to gradient-based optimization. Online batch selection, however, must be more tightly coupled with optimization utility, since its immediate effect is to modulate gradient updates rather than to alter the composition of the training dataset. As a result, online batch selection prioritizes signals that reflect potential loss reduction and training dynamics, making it fundamentally different from classical active learning in both motivation and mechanism.

C. Additional Results

C.1. Parameter Analysis

Impact of memory buffer size and low projected dimension. We evaluate the influence of the projected dimensions d_1 , d_2 , and the memory buffer size M on both model performance and resource usage. As shown in Figures 4 and 6, increasing the projected dimensions and memory buffer consistently improves accuracy, which gradually saturates once d_1 , d_2 , and M become sufficiently large. The trends of d_1 and d_2 are consistent with the Johnson-Lindenstrauss lemma (Johnson et al., 1984; Ailon & Chazelle, 2006; Jin et al., 2021; Ailon & Chazelle, 2009; Tropp, 2011) and related random-projection-inspired efficient learning methods (Li et al., 2026; Zou et al., 2025b;a;c), while a sufficiently large M leads to performance saturation, indicating that the stored representations are adequate to capture global sample diversity. The corresponding increases in memory usage and computation time are minor, and arise only when computing inter-sample diversity.

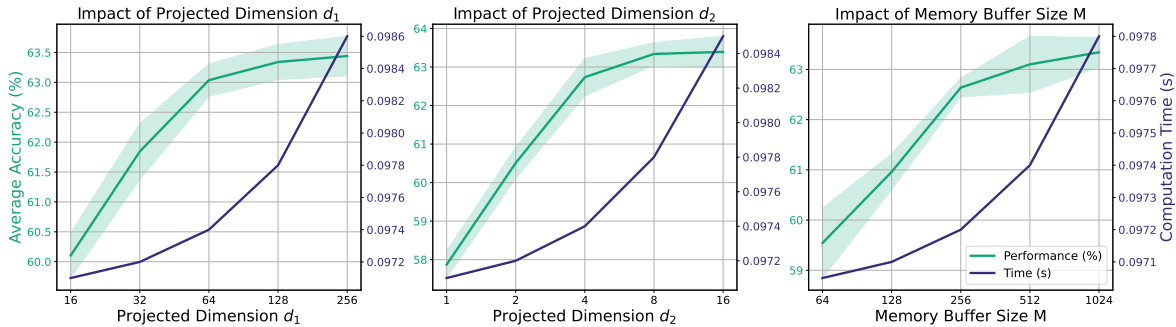


Figure 6. **Impact of low projected dimensions d_1 and d_2 , and memory buffer size M on model performance and additional computation time for Qwen-2.5 on MMLU.** The green curves show average accuracy (%), and the blue curves show additional computation time per batch (s). Performance improves with increasing projected dimensions and memory buffer size, while the additional computation time remains negligible.

In Figure 4, increasing d_1 , d_2 , and M has no significant impact on extra memory usage. The base overhead of around 1.17GB comes from the projection matrices Γ_1 and Γ_2 , which is small compared to the overall training memory footprint of about 22GB. In Figure 6, the additional time per batch is also negligible relative to the total training cost of 2.3s per batch.

Overall, the extra resource cost is minimal compared to the performance gains. In practice, parameter selection only needs to ensure that d_1 , d_2 , and M are sufficiently large for stable performance.

FLOPs breakdown of UDS. The throughput comparison in Table 3 reflects end-to-end training speed, but UDS also differs from full fine-tuning because it performs backpropagation only on the selected K samples in each batch. To isolate the extra computation introduced by the selection mechanism itself, we further report a FLOPs breakdown of UDS in Table 5. For each batch, UDS consists of three stages: (1) a forward pass over all B candidate samples to obtain logit matrices, (2) selection computation for s_{intra} and s_{inter} , and (3) backpropagation over the selected K samples. The selection stage contributes only a small fraction of total FLOPs, roughly 0.5%–2.6% across settings, indicating that the core scoring mechanism introduces limited additional computation.

Table 5. **FLOPs Breakdown During UDS Training.** We report the FLOPs of the forward pass, selection computation, and backward pass for the settings in Table 3.

Model	Stage	MMLU	ScienceQA	GSM8K	CodeAlpaca
Llama-3.1-8B	Forward	3.5×10^{17}	1.1×10^{17}	1.3×10^{16}	4.0×10^{16}
	Selection	9.2×10^{15}	1.4×10^{15}	2.1×10^{14}	5.7×10^{14}
	Backward	3.5×10^{17}	1.1×10^{17}	1.3×10^{16}	4.0×10^{16}
Qwen-2.5-7B	Forward	3.4×10^{17}	1.0×10^{17}	1.4×10^{16}	3.9×10^{16}
	Selection	1.1×10^{16}	1.7×10^{15}	3.0×10^{14}	6.8×10^{14}
	Backward	8.5×10^{16}	1.0×10^{17}	7.1×10^{15}	3.9×10^{16}

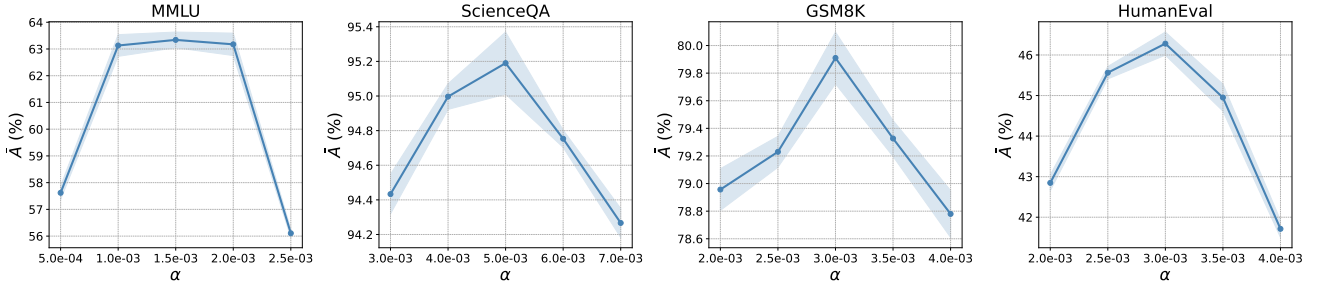


Figure 7. **Sensitivity analysis of the trade-off factor α on Qwen-2.5-7B.** Results are reported across four benchmarks—MMLU, ScienceQA, GSM8K, and HumanEval—using the data selection fractions specified in Table 6.

Impact of trade-off factor α . Figure 7 presents the sensitivity analysis of α on Qwen-2.5-7B across four datasets. The results consistently exhibit an inverted U-shaped trend: performance improves as α increases, peaks at an optimal value, and then declines as α continues to grow. This pattern highlights the importance of maintaining a proper balance. The optimal α values corresponding to the selected data fractions in Table 6 are summarized in Table 7.

Table 6. **Data selection fractions across benchmark datasets.** Percentages of samples selected from MMLU, ScienceQA, GSM8K, and CodeAlpaca, evaluated with Llama-3.1-8B and Qwen-2.5-7B as backbone models.

Model	MMLU	ScienceQA	GSM8K	CodeAlpaca
Llama-3.1-8B	50%	50%	50%	50%
Qwen-2.5-7B	12.5%	50%	25%	50%

Table 7. **Optimal trade-off factor α across datasets and backbones.** Optimal α values for MMLU, ScienceQA, GSM8K, and CodeAlpaca using Llama-3.1-8B and Qwen-2.5-7B, with data selection fractions given in Table 6.

Model	MMLU	ScienceQA	GSM8K	CodeAlpaca
Llama-3.1-8B	4.5×10^{-3}	7×10^{-3}	1×10^{-3}	5×10^{-3}
Qwen-2.5-7B	1.5×10^{-3}	5×10^{-3}	3×10^{-3}	3×10^{-3}

C.2. Additional Ablation Studies on Buffer Update Policies, Distance Aggregation Strategies, and Random Matrix Constructions

For buffer update policies, we compare our default FIFO strategy with two alternatives: reservoir sampling (random replacement within the buffer) and class-aware sampling (constructing class prototypes via K-means). Experiments on

the MMLU benchmark using Qwen-2.5-7B (Table 8) show that all three policies yield comparable performance, with no notable differences observed.

For distance aggregation strategies, we evaluate our average-distance scheme against two variants—farthest- k and soft-min—again on MMLU with Qwen-2.5-7B (Table 9). The results similarly indicate no significant performance differences among these methods.

In UDS, we employ an SRFT-style construction for the bidirectional random projection matrix. To assess its necessity, we compare it against two alternative constructions: Sparse JL and CountSketch-style tensor sketches. Experiments on MMLU with Qwen-2.5-7B (Table 10) show that the CountSketch-style transform achieves performance nearly identical to the SRFT-style transform, whereas Sparse JL performs significantly worse.

The inferior performance of Sparse JL stems from its failure to satisfy the JL lemma under the bidirectional formulation, similar to the Gaussian-style construction discussed in Section 3.2. The issue arises because the Kronecker product disrupts the independence structure of the random variables, thereby violating the distance-preserving requirements of the JL lemma. In contrast, both SRFT-style and CountSketch-style constructions continue to satisfy the JL lemma when used bidirectionally.

From an efficiency perspective, decomposing the full random projection into a bidirectional one already greatly reduces memory usage and computation cost, making these overheads negligible relative to other components of the algorithm. Consequently, as long as a construction satisfies the JL lemma in the bidirectional setting, it is unnecessary to further distinguish which one is marginally more efficient. The SRFT-style construction is therefore sufficient for our purposes, and is what we adopt in our main experiments.

Table 8. **Ablation Studies on Buffer Update Policies.** We report average accuracy on MMLU using Qwen-2.5-7B.

FIFO	Reservoir Sampling	Class-aware Sampling
63.34 $\pm$ 0.36	63.45 $\pm$ 0.22	63.15 $\pm$ 0.50

Table 9. **Ablation Studies on Distance Aggregation Policies.** We report average accuracy on MMLU using Qwen-2.5-7B.

Average Distance	Farthest-k	Soft-min
63.34 $\pm$ 0.36	63.18 $\pm$ 0.29	62.98 $\pm$ 0.48

Table 10. **Ablation Studies on Different Construction of the Random Matrix.** We report average accuracy on MMLU using Qwen-2.5-7B.

SRFT-style	Sparse JL	CountSketch-style
63.34 $\pm$ 0.36	55.28 $\pm$ 0.65	63.12 $\pm$ 0.31

C.3. Experiments on Full SFT, Larger Batch Sizes, and Instruction-Tuned Models

To further demonstrate the scalability of UDS, we provide additional results under three settings: larger batch sizes (mini-batch size $B = 16$ in Table 11), full-parameter finetuning with Qwen-2.5-7B and Qwen-2.5-14B (Table 12), and instruction-tuned models using Qwen-2.5-7B-Instruct (Table 14). Across all scenarios, UDS consistently outperforms the corresponding baselines, strengthening the robustness and generalization capability of the method.

Why UDS outperforms full fine-tuning with less data. We further analyze why UDS can outperform full fine-tuning while using only a fraction of the training data. We hypothesize that full-data fine-tuning may spend a considerable portion of the training budget on redundant or stage-misaligned samples, which can dilute effective updates and hurt generalization. To verify this, we analyze the selection behavior of UDS on Llama-3.1-8B with MMLU under a 50% selection ratio. Since MMLU is trained for only one epoch, we first run regular full fine-tuning, record each sample’s loss before (ℓ_{init}) and after (ℓ_{final}) training, and classify all samples into the four categories in Table 2 using the corresponding median losses as thresholds. We then run UDS and compare the category distribution between selected and non-selected samples.

As shown in Table 13, the selected samples are predominantly high-utility samples, while the non-selected samples are disproportionately too easy or too hard. This indicates that UDS concentrates the training budget on learnable and high-value samples, which helps explain why it can outperform full fine-tuning despite using less data.

Table 11. Accuracy Comparison for Different Online Batch Selection Methods with Larger Batch Size. We evaluate various methods across four benchmarks: MMLU, ScienceQA, GSM8K, and HumanEval. $\bar{A}$ denotes average accuracy or Pass@1 reported in percentage (%). The best results of $\bar{A}$ are highlighted in **bold**.

Model	Method	MMLU	ScienceQA	GSM8K	HumanEval
Qwen-2.5-7B	Regular	54.44 $\pm$ 0.67	94.68 $\pm$ 0.25	78.76 $\pm$ 0.48	45.12 $\pm$ 0.64
	Random	56.35 $\pm$ 1.54	94.40 $\pm$ 0.19	78.24 $\pm$ 0.53	41.68 $\pm$ 0.85
	MaxLoss	55.76 $\pm$ 1.13	94.08 $\pm$ 0.34	78.28 $\pm$ 0.22	42.06 $\pm$ 0.47
	MaxGrad	56.81 $\pm$ 0.85	93.97 $\pm$ 0.21	78.44 $\pm$ 0.25	41.94 $\pm$ 0.79
	RHO-Loss	58.28 $\pm$ 0.94	94.45 $\pm$ 0.42	78.81 $\pm$ 0.43	44.34 $\pm$ 0.68
	GREATS	60.53 $\pm$ 1.27	94.72 $\pm$ 0.13	79.02 $\pm$ 0.07	45.13 $\pm$ 0.41
	UDS (Ours)	62.71$\pm$0.88	95.08$\pm$0.27	79.38$\pm$0.25	45.98$\pm$0.59

Table 12. Accuracy Comparison for Different Online Batch Selection Methods with Full Finetuning. We evaluate various methods with Qwen-2.5-7B and Qwen-2.5-14B across four benchmarks: MMLU, ScienceQA, GSM8K, and HumanEval. $\bar{A}$ denotes average accuracy or Pass@1 reported in percentage (%). The best results of $\bar{A}$ for each backbone are highlighted in **bold**.

Model	Method	MMLU	ScienceQA	GSM8K	HumanEval
Qwen-2.5-7B	Regular	55.64 $\pm$ 0.43	94.52 $\pm$ 0.12	77.56 $\pm$ 0.16	48.03 $\pm$ 0.33
	Random	56.05 $\pm$ 0.71	94.38 $\pm$ 0.25	76.95 $\pm$ 0.35	41.46 $\pm$ 0.56
	MaxLoss	55.59 $\pm$ 0.27	94.41 $\pm$ 0.19	76.82 $\pm$ 0.42	42.36 $\pm$ 0.60
	MaxGrad	55.16 $\pm$ 0.61	94.27 $\pm$ 0.32	77.08 $\pm$ 0.25	43.29 $\pm$ 0.52
	RHO-Loss	57.94 $\pm$ 1.73	94.64 $\pm$ 0.21	77.51 $\pm$ 0.36	46.09 $\pm$ 0.40
	GREATS	58.86 $\pm$ 0.54	94.75 $\pm$ 0.23	77.86 $\pm$ 0.17	47.56 $\pm$ 0.49
	UDS (Ours)	63.27$\pm$0.33	95.06$\pm$0.28	78.26$\pm$0.39	48.44$\pm$0.70
Qwen-2.5-14B	Regular	63.72 $\pm$ 0.51	95.86 $\pm$ 0.24	83.66 $\pm$ 0.21	49.36 $\pm$ 0.33
	Random	62.39 $\pm$ 0.83	94.91 $\pm$ 0.51	83.14 $\pm$ 0.27	47.52 $\pm$ 0.25
	MaxLoss	62.68 $\pm$ 0.74	94.85 $\pm$ 0.31	83.25 $\pm$ 0.41	48.12 $\pm$ 0.62
	MaxGrad	62.55 $\pm$ 0.61	95.12 $\pm$ 0.42	83.10 $\pm$ 0.38	47.85 $\pm$ 0.55
	RHO-Loss	64.21 $\pm$ 0.45	95.25 $\pm$ 0.28	83.85 $\pm$ 0.32	48.65 $\pm$ 0.48
	GREATS	64.85 $\pm$ 0.53	95.54 $\pm$ 0.35	84.12 $\pm$ 0.35	49.05 $\pm$ 0.51
	UDS (Ours)	65.86$\pm$0.79	96.33$\pm$0.19	84.57$\pm$0.26	50.30$\pm$0.47

Table 13. Category Distribution of Selected and Non-selected Samples. We report the percentage of each category on Llama-3.1-8B with MMLU under a 50% selection ratio.

Category	Selected Samples	Non-selected Samples
High Utility (High $\rightarrow$ Low)	55.7%	11.5%
Too Easy (Low $\rightarrow$ Low)	16.9%	38.1%
Too Hard (High $\rightarrow$ High)	19.1%	34.5%
Overfitted (Low $\rightarrow$ High)	8.3%	15.9%

Table 14. Accuracy Comparison for Different Online Batch Selection Methods on Instruction Tuning Model. We evaluate various methods across four benchmarks: MMLU, ScienceQA, GSM8K, and HumanEval. $\bar{A}$ denotes average accuracy or Pass@1 reported in percentage (%). The best results of $\bar{A}$ are highlighted in **bold**.

Model	Method	MMLU	ScienceQA	GSM8K	HumanEval
Qwen-2.5-7B-Instruct	Regular	50.15 $\pm$ 0.15	94.47 $\pm$ 0.18	75.74 $\pm$ 0.19	45.29 $\pm$ 0.37
	Random	47.38 $\pm$ 1.41	94.60 $\pm$ 0.32	75.36 $\pm$ 0.29	41.09 $\pm$ 0.64
	MaxLoss	47.92 $\pm$ 0.87	94.92 $\pm$ 0.53	75.57 $\pm$ 0.42	41.17 $\pm$ 0.41
	MaxGrad	47.63 $\pm$ 0.51	94.46 $\pm$ 0.36	75.63 $\pm$ 0.51	41.24 $\pm$ 0.38
	RHO-Loss	49.62 $\pm$ 0.81	95.14 $\pm$ 0.26	76.15 $\pm$ 0.26	43.19 $\pm$ 0.23
	GREATS	52.69 $\pm$ 1.03	95.22 $\pm$ 0.19	76.20 $\pm$ 0.35	44.36 $\pm$ 0.52
	UDS (Ours)	53.86$\pm$0.42	95.56$\pm$0.14	76.80$\pm$0.37	45.75$\pm$0.28

C.4. Additional Experiments on Tasks Requiring Long-Context Reasoning

In all previous main experiments, we set the maximum sequence length to 512. To assess whether UDS can effectively capture utility and diversity for training samples that require long-context reasoning, we further conduct experiments on the MATH dataset (Hendrycks et al., 2021b), using maximum sequence lengths of 2048 and 32768. We train on the MATH training split and evaluate on its test split. As shown in Table 15, UDS continues to deliver strong performance compared with other baselines, even under substantially longer context windows.

We also examine the system-level efficiency of UDS under long contexts. Let the sequence length be N . The SVD computation for the logits matrix scales quadratically with sequence length in our setting, i.e., $O(N^2)$. Although this may appear costly, the attention mechanism also scales as $O(N^2)$ and dominates the forward and backward passes at long sequence lengths. Therefore, the relative overhead of the SVD step does not grow disproportionately as the context length increases. Empirically, the SVD computation accounts for only about 2–3% of the total training time at both sequence lengths of 2048 and 32768.

Table 15. **Additional Studies on MATH dataset with Longer CoT Reasoning.** We report average accuracy using Qwen-2.5-7B under different maximum sequence lengths.

Max Seq. Len.	Regular	Random	MaxLoss	MaxGrad	RHO-Loss	GREATS	UDS (Ours)
2048	45.89 $\pm$ 0.14	42.85 $\pm$ 0.28	42.69 $\pm$ 0.48	42.77 $\pm$ 0.61	45.35 $\pm$ 0.32	45.66 $\pm$ 0.29	46.27$\pm$0.35
32768	60.92 $\pm$ 0.16	57.81 $\pm$ 0.30	57.65 $\pm$ 0.45	57.74 $\pm$ 0.58	60.38 $\pm$ 0.35	60.71 $\pm$ 0.27	61.35$\pm$0.32

C.5. Additional Experiments on OOD Evaluation

To further assess the robustness of UDS, we conduct out-of-distribution (OOD) evaluations using GSM8K as a case study. Specifically, we train on the GSM8K training set and evaluate on its in-distribution test set as well as two OOD benchmarks: MATH500 and AMC23 (Lightman et al., 2023). As shown in Table 16, UDS continues to exhibit strong performance across both in-distribution and OOD settings, demonstrating its robustness to distribution shifts.

Table 16. **Accuracy Comparison for Different Online Batch Selection Methods on OOD Datasets.** We evaluate various methods across three benchmarks: GSM8K (in-distribution), MATH500, and AMC23 (out-of-distribution). $\bar{A}$ denotes average accuracy or Pass@1 reported in percentage (%). The best results of $\bar{A}$ are highlighted in **bold**.

Model	Method	GSM8K	MATH500	AMC23
Qwen-2.5-7B	Regular	78.23 $\pm$ 0.07	41.83 $\pm$ 0.38	24.67 $\pm$ 0.73
	Random	77.69 $\pm$ 0.29	41.09 $\pm$ 0.24	24.39 $\pm$ 0.64
	MaxLoss	77.78 $\pm$ 0.36	41.18 $\pm$ 0.34	23.89 $\pm$ 1.12
	MaxGrad	77.62 $\pm$ 0.28	41.13 $\pm$ 0.27	24.26 $\pm$ 0.58
	RHO-Loss	78.38 $\pm$ 0.43	41.84 $\pm$ 0.19	25.59 $\pm$ 0.69
	GREATS	78.61 $\pm$ 0.41	42.27 $\pm$ 0.39	25.97 $\pm$ 0.76
	UDS (Ours)	79.91$\pm$0.23	42.56$\pm$0.74	26.38$\pm$0.83

C.6. Comparison with Offline Data Selection Methods

We additionally include a representative offline data selection method, FisherSFT (Deb et al., 2025), which uses Fisher information gain to select a training subset, in our comparison. We conduct experiments on all four datasets using Qwen-2.5-7B as the backbone model. Since the embedding dimension of Qwen-2.5-7B is much larger than that of the GPT-2 model used in the original FisherSFT paper, directly applying FisherSFT leads to out-of-memory errors in our hardware environment. We therefore apply a random down-projection, similar in spirit to UDS, to reduce the representation dimension to 1024, which is the maximum feasible dimension on a single NVIDIA RTX 3090 in our setup. We refer to this adapted implementation as FisherSFT in the following comparison.

Since FisherSFT is an offline method that performs a single forward pass over the entire dataset and selects data before training, while UDS is an online method that scores samples during training, directly comparing their total wall-clock time is not fully meaningful. We therefore compare their algorithm-specific overhead per epoch. For UDS, this is the cumulative time spent computing $s_{\text{intra}} + \alpha \cdot s_{\text{inter}}$ after the forward pass. For FisherSFT, this is the time spent on greedy subset selection

after embeddings are computed. Both FisherSFT and UDS select the same fraction of data, and all other training details remain the same. The results in Table 17 show that UDS achieves better final performance than FisherSFT, while Tables 18 and 19 show that UDS is also more efficient in both algorithm-specific time and peak GPU memory consumption.

Table 17. **Accuracy Comparison with Offline Data Selection Strategy FisherSFT.** We report average accuracy using Qwen-2.5-7B.

Method	MMLU	ScienceQA	GSM8K	HumanEval
FisherSFT	57.85 $\pm$ 0.62	94.02 $\pm$ 0.34	78.35 $\pm$ 0.30	43.87 $\pm$ 0.59
UDS (Ours)	63.34$\pm$0.36	95.19$\pm$0.22	79.91$\pm$0.23	46.28$\pm$0.35

Table 18. **Algorithm-specific Time of FisherSFT and UDS.** We report wall-clock seconds per epoch spent on each method’s selection-specific computation.

Method	MMLU	ScienceQA	GSM8K	HumanEval
FisherSFT	635	1672	1281	10107
UDS (Ours)	584	186	463	645

Table 19. **Peak GPU Memory Consumption of FisherSFT and UDS.** We report peak GPU memory consumption in GB under the same hardware setup.

Method	MMLU	ScienceQA	GSM8K	HumanEval
FisherSFT	22.9	23.0	22.0	22.8
UDS (Ours)	19.2	19.2	18.0	19.4

C.7. Analysis on Considering Within-Batch Diversity

We conducted an additional experiment showing that incorporating diversity calculation within the batch yields no notable difference in accuracy. This is because the memory buffer size is significantly larger than the batch size; thus, it is less likely that a similar sample appears in the buffer while being absent from the candidate batch. This comparison follows the experimental setting in Table 3 using Qwen-2.5-7B.

D. Theoretical Analysis

D.1. Relationship Between Frobenius Norm and Nuclear Norm

Lemma 3.1 establishes a fundamental relationship between two widely used matrix norms: the Frobenius norm $\|\cdot\|_F$ and the nuclear norm $\|\cdot\|_*$. Recall that if $\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t) \in \mathbb{R}^{N \times V}$ has singular values $\{\sigma_j\}_{j=1}^r$ with rank $r \leq \min(N, V)$, then

$$\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_F = \left(\sum_{j=1}^r \sigma_j^2 \right)^{1/2}, \quad \|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_* = \sum_{j=1}^r \sigma_j. \quad (13)$$

The two-sided inequality in Lemma 3.1 is proved as follows.

(1) *Lower bound.* Since the Euclidean norm of a set of nonnegative numbers is always no larger than their sum, we have

$$\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_F = \left(\sum_{j=1}^r \sigma_j^2 \right)^{1/2} \leq \sum_{j=1}^r \sigma_j = \|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_*. \quad (14)$$

Equality holds iff at most one singular value is nonzero, i.e., $\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$ is rank-1 (in which case the Euclidean norm and the sum of singular values coincide).

(2) *Upper bound.* Applying the Cauchy–Schwarz inequality to the vectors $(1, \dots, 1) \in \mathbb{R}^r$ and $(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^r$ yields

$$\|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_* = \sum_{j=1}^r \sigma_j \leq \left(\sum_{j=1}^r 1^2 \right)^{1/2} \left(\sum_{j=1}^r \sigma_j^2 \right)^{1/2} = \sqrt{r} \|\mathbf{L}(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_F. \quad (15)$$

Table 20. Accuracy Comparison With and Without Within-batch Diversity. We report the average accuracy using Qwen-2.5-7B.

Method	MMLU	ScienceQA	GSM8K	HumanEval
w/o within-batch diversity	63.34 $\pm$ 0.36	95.19 $\pm$ 0.22	79.91 $\pm$ 0.23	46.28 $\pm$ 0.35
w/ within-batch diversity	63.28 $\pm$ 0.34	95.26 $\pm$ 0.19	79.80 $\pm$ 0.38	46.39 $\pm$ 0.42

Since $r \leq \min(N, V)$ we obtain the stated upper bound

$$\|L(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_* \leq \sqrt{\min(N, V)} \|L(\mathbf{x}_t^i; \boldsymbol{\theta}_t)\|_F. \quad (16)$$

Equality in the Cauchy–Schwarz step holds iff the singular-value vector is proportional to the all-ones vector, i.e. all nonzero singular values are equal. Thus the rightmost equality is achieved when $r = \min(N, V)$ (full possible rank) and the r singular values are equal.

Implications. This lemma highlights that the nuclear norm and Frobenius norm, though correlated, capture different structural aspects of $L(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$. The Frobenius norm measures the overall magnitude of logits, while the nuclear norm is also sensitive to their rank structure. A large nuclear norm relative to the Frobenius norm implies more “spread-out” singular values and thus higher intra-sample diversity.

D.2. Optimization Analysis using Adam

When using the Adam (Kingma & Ba, 2014) as optimizer, the parameter update at step t is

$$\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t = -\eta_t \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon_{\text{adam}}}}, \quad (17)$$

where $\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$ are the bias-corrected first and second moment estimates of the stochastic gradients, respectively:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_t^2, \quad (18)$$

with $\mathbf{g}_t = \frac{1}{B} \sum_{i=1}^B \nabla_{\boldsymbol{\theta}} \ell(\mathbf{x}_t^i; \boldsymbol{\theta}_t)$.

After the parameter update $\boldsymbol{\theta}_t \rightarrow \boldsymbol{\theta}_{t+1}$, the logits matrix for datapoint $\mathbf{x}_t^i$ can be expanded using a first-order Taylor expansion:

$$L(\mathbf{x}_t^i; \boldsymbol{\theta}_{t+1}) \approx L(\mathbf{x}_t^i; \boldsymbol{\theta}_t) + \nabla_{\boldsymbol{\theta}} L(\mathbf{x}; \boldsymbol{\theta}_t) \cdot (\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t), \quad (19)$$

so that the induced change is

$$\delta L(\mathbf{x}; \boldsymbol{\theta}_t) \approx -\eta_t \nabla_{\boldsymbol{\theta}} L(\mathbf{x}; \boldsymbol{\theta}_t) \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon_{\text{adam}}}}. \quad (20)$$

D.3. Distance Preservation under Low-dimensional Projection

We show that our two-sided projection construction can be algebraically reduced to a single subsampled randomized Fourier transform (SRFT) on $\mathbb{R}^{NV}$. Recall the standard property of the vec operator with Kronecker products: for conformable matrices $\mathbf{A}, \mathbf{B}, \mathbf{X}$, we have

$$\text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = (\mathbf{B}^\top \otimes \mathbf{A}) \text{vec}(\mathbf{X}). \quad (21)$$

Applying this to $\boldsymbol{\Gamma}_2 \in \mathbb{R}^{d_2 \times N}$, $\boldsymbol{\Gamma}_1 \in \mathbb{R}^{d_1 \times V}$, and $L(\mathbf{x}_t^i; \boldsymbol{\theta}_t) \in \mathbb{R}^{N \times V}$, we obtain

$$\mathbf{z}_t^i = \text{vec}(\boldsymbol{\Gamma}_2 L(\mathbf{x}_t^i; \boldsymbol{\theta}_t) \boldsymbol{\Gamma}_1^\top) = (\boldsymbol{\Gamma}_1 \otimes \boldsymbol{\Gamma}_2) \text{vec}(L(\mathbf{x}_t^i; \boldsymbol{\theta}_t)) = \boldsymbol{\Gamma} \text{vec}(L(\mathbf{x}_t^i; \boldsymbol{\theta}_t)), \quad (22)$$

where $\boldsymbol{\Gamma} = \boldsymbol{\Gamma}_1 \otimes \boldsymbol{\Gamma}_2 \in \mathbb{R}^{d_1 d_2 \times NV}$. Using the Kronecker identity $(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{A}\mathbf{C}) \otimes (\mathbf{B}\mathbf{D})$, we can rewrite

$$\boldsymbol{\Gamma} = \left(\sqrt{\frac{V}{d_1}} \mathbf{S}_1 \mathbf{F}_1 \mathbf{D}_1 \right) \otimes \left(\sqrt{\frac{N}{d_2}} \mathbf{S}_2 \mathbf{F}_2 \mathbf{D}_2 \right) \quad (23)$$

$$= \sqrt{\frac{NV}{d_1 d_2}} (\mathbf{S}_1 \otimes \mathbf{S}_2) (\mathbf{F}_1 \otimes \mathbf{F}_2) (\mathbf{D}_1 \otimes \mathbf{D}_2), \quad (24)$$

where $S = S_1 \otimes S_2$, $F = F_1 \otimes F_2$, and $D = D_1 \otimes D_2$. Since each F_i is orthonormal and D_i has independent Rademacher entries, F is orthonormal and D remains a diagonal matrix with independent Rademacher entries. Hence, $\Gamma = \sqrt{\frac{NV}{d_1 d_2}} S F D$ is exactly an SRFT.

Remark: This Kronecker structure preserves the SRFT type. In contrast, if Γ_1, Γ_2 are i.i.d. Gaussian (as in classical JL), then $\Gamma_1 \otimes \Gamma_2$ no longer has i.i.d. Gaussian entries due to induced correlations.

Johnson-Lindenstrauss Guarantee. Let $u_1, \dots, u_n \in \mathbb{R}^{NV}$ denote n vectors of interest. Consider all pairwise differences $u_i - u_j$, $1 \leq i < j \leq n$; there are at most $n(n-1)/2$ such vectors. By standard results on Fast JL transforms (SRFT/FJLT) (Ailon & Chazelle, 2006; Jin et al., 2021; Ailon & Chazelle, 2009; Tropp, 2011), for any $0 < \epsilon < 1$ and a fixed set of n vectors in $\mathbb{R}^{NV}$, a random SRFT $\Gamma \in \mathbb{R}^{d_1 d_2 \times NV}$ preserves the ℓ_2 norms up to $(1 \pm \epsilon)$ with high probability provided

$$d_1 d_2 \gtrsim C \epsilon^{-2} \log(NV) \text{polylog}(n), \quad (25)$$

where C is an absolute constant and the polylog factor depends on the SRFT construction. This gives that, with high probability, for all i, j :

$$(1 - \epsilon) \|u_i - u_j\|_2^2 \leq \|\Gamma(u_i - u_j)\|_2^2 \leq (1 + \epsilon) \|u_i - u_j\|_2^2, \quad (26)$$

which is exactly the claimed Johnson-Lindenstrauss distance preservation.

E. Detailed Experimental Setting

E.1. Datasets

We conduct our experiments on the following datasets, with important information listed in Table 21. Typically, for each reported result, we train the model on a single training set and evaluate it on a single test set. Specifically, for MMLU, models were trained on MMLU’s auxiliary dataset and tested on its test set; for GSM8K, models were trained on GSM8K’s training set and tested on its test set; for ScienceQA, models were trained on its training set and tested on its test set; for HumanEval, models were trained on CodeAlpaca and evaluated on the HumanEval benchmark; and for MATH, models were trained on MATH’s training set and tested on its test set. Specifically, for the OOD experiments, we evaluate the model trained on GSM8K directly on the MATH500 and AMC23 datasets. Brief introduction for each dataset are provided in the following:

- **MMLU** (Hendrycks et al., 2021a) is a benchmark designed to assess general knowledge understanding, consisting of multiple-choice questions from various branches of knowledge. It covers 57 tasks including elementary mathematics, US history, computer science, law, and more.
- **ScienceQA** (Lu et al., 2022) is a multimodal science question answering dataset collected from elementary and high school science curricula, including multiple-choice problems that align with California Common Core Content Standards. The questions in dataset are sourced from open resources managed by IXL Learning, an online learning platform curated by experts in the field of K-12 education. We extract textual parts within it following (Li et al., 2024).
- **GSM8K** (Cobbe et al., 2021) is a dataset of 8.5K high quality linguistically diverse grade school math word problems, each requiring multi-step arithmetic reasoning. Answers are provided in a step-by-step explanation format.
- **CodeAlpaca-20k** (Chaudhary, 2023) is an instruction-following dataset consisting of 20k examples generated to align code-related prompts with helpful outputs. It is synthetically constructed to enhance code instruction tuning.
- **HumanEval** (Chen et al., 2021) is a code generation benchmark including 164 Python programming problems. The dataset was handwritten to ensure not to be included in the training set of code generation models.
- **MATH** (Hendrycks et al., 2021b) consists of 12.5K challenging competition mathematics problems from high school math competitions. It covers diverse subjects including algebra, geometry, counting, and calculus, requiring complex heuristic reasoning to solve.
- **MATH500** (Lightman et al., 2023) is a curated subset of 500 examples from the MATH dataset’s test split. It is widely used to evaluate the mathematical reasoning capabilities of LLMs in a more computationally efficient manner while maintaining the difficulty distribution of the original benchmark.

- **AMC23** (Lightman et al., 2023) comprises problems from the 2023 American Mathematics Competitions (AMC 12). It serves as an out-of-distribution (OOD) benchmark to assess the model’s ability to generalize to recent mathematical problems that were not seen during the training phase.

Table 21. **Details of MMLU, ScienceQA, GSM8K, CodeAlpaca, HumanEval, MATH, MATH500 and AMC23 Datasets.** We list the number of training and testing samples and task types for the following datasets used in our experiments.

Dataset	Training Samples	Testing Samples	Task Types
MMLU (Hendrycks et al., 2021a)	99842	14042	Multiple Choice
ScienceQA (Lu et al., 2022)	12726	4241	Multiple Choice
GSM8K (Cobbe et al., 2021)	7473	1319	Math Problems
CodeAlpaca-20k (Chaudhary, 2023)	20022	—	Code Instruction
HumanEval (Chen et al., 2021)	—	164	Code Generation
MATH (Hendrycks et al., 2021b)	7500	5000	Math Problems
MATH500 (Lightman et al., 2023)	—	500	Math Problems
AMC23 (Lightman et al., 2023)	—	40	Math Problems

E.2. Training Configuration

In this section, we detail the experiment configuration with hyper parameters. General configurations across all backbones and datasets are listed in Table 22. Dataset-specific and model-specific setting are listed in Table 23. For all baseline approaches, we adopt the same configurations described above. For GREATS, we set the validation set size to 5. For RHO-Loss, we use Llama-2-13B as the reference model for Llama-3.1-8B, and Qwen-2.5-14B as the reference model for Qwen-2.5-7B. Ideally, Llama-3.1-70B would serve as a better reference model for Llama-3.1-8B, but its computational cost makes it impractical in our setting. Fortunately, we find that Llama-2-13B provides sufficiently strong guidance, and thus we adopt it as a surrogate reference model.

Table 22. **General Training Hyperparameters with LoRA.** Shared configuration across all experiments, including rank settings, optimizer details, and architectural choices.

Parameter	Value
Rank (r)	8
Scaling factor (α)	16
Target modules	{q, k, v, o, gate, down, up}-proj
Optimizer	AdamW
Warmup ratio	0.01
Gradient accumulated batch	128
Dropout rate	0.00

Table 23. **Dataset-Specific and Model-Specific Training Configurations during training.** Task-optimized settings for Llama-3.1-8B and Qwen-2.5-7B across four benchmarks, showing variations in epoch counts, learning rates, and sequence lengths based on dataset characteristics and model requirements.

Model	Parameter	MMLU	ScienceQA	GSM8K	CodeAlpaca
Llama-3.1-8B	Epochs	1	20	1	2
	Learning rate	1.5×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
	Max sequence length	512	256	512	512
	micro batch size	8	8	8	8
Qwen-2.5-7B	Epochs	1	20	1	2
	Learning rate	1.5×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
	Max sequence length	512	256	512	512
	micro batch size	8	8	8	8

F. Limitations and Future Work

Limitations. One potential limitation of our framework lies in the computation of the nuclear norm. While it can be obtained directly from the singular values of the logits matrix without incurring additional expensive backpropagation, the SVD of large matrices may still introduce non-negligible overhead in practice, particularly for long sequences with large vocabulary sizes. We also explored approximate methods such as randomized SVD to accelerate the computation, but observed that the loss in precision often degrades the overall performance. Another limitation is the difficulty of providing a rigorous theoretical proof for the linear correlation between the nuclear norm of the logits matrix and the resulting loss reduction or effective matrix rank given the complexity of the large model.

Future Work. An important future direction is to explore more efficient and accurate estimators of the nuclear norm. Potential avenues include exploiting structured low-rank approximations or sparsity-aware computation ([Sun et al., 2026](#)), leveraging iterative or block-wise SVD techniques, or designing surrogate scoring functions that preserve the optimization and diversity signals of the nuclear norm while being cheaper to compute. Such improvements would further enhance the scalability of our framework to larger models and longer input sequences.